

Q.No:6

Question description

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a,b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a,b]$.

Input

The first input line has two integers n and q : the size of the array and the number of queries.

The next line has n values $t_1, t_2, \dots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b " or "2 a b ".

Output

Print the answer to each sum query.

Constraints

- $1 \leq n, q \leq 2 \cdot 10^5$
- $1 \leq t_i \leq 10^6$
- $1 \leq a \leq b \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
```

EXPECTED OUTPUT

```
9
```

Test Case 2

INPUT (STDIN)

```
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
```

EXPECTED OUTPUT

```
17
32
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
void build(int k,int
l,int r)
```

Code:

```
#include <stdio.h>

#define MAX 200005

long long tree[4 * MAX];
long long lazyA[4 * MAX];
long long lazyB[4 * MAX];
long long arr[MAX];
```

```
void build(int k, int l, int r)
{
    if (l == r)
    {
        tree[k] = arr[l];
        return;
    }

    int mid = (l + r) / 2;

    build(2 * k, l, mid);
    build(2 * k + 1, mid + 1, r);

    tree[k] = tree[2 * k] + tree[2 * k + 1];
}

void apply(int k, int l, int r, long long a, long long b)
{
    long long len = r - l + 1;
    long long sumIndex = (long long)(l + r) * len / 2;

    tree[k] += a * sumIndex + b * len;
    lazyA[k] += a;
    lazyB[k] += b;
}

void push(int k, int l, int r)
{
    if (lazyA[k] == 0 && lazyB[k] == 0)
        return;

    if (l != r)
    {
        int mid = (l + r) / 2;

        apply(2 * k, l, mid, lazyA[k], lazyB[k]);
        apply(2 * k + 1, mid + 1, r, lazyA[k], lazyB[k]);
    }

    lazyA[k] = 0;
    lazyB[k] = 0;
}

void update(int k, int l, int r, int ql, int qr)
{
    if (ql <= l && r <= qr)
    {
        long long a = 1;
        long long b = 1 - ql;

        apply(k, l, r, a, b);
        return;
    }
}
```

```
push(k, l, r);

int mid = (l + r) / 2;

if (ql <= mid)
    update(2 * k, l, mid, ql, qr);

if (qr > mid)
    update(2 * k + 1, mid + 1, r, ql, qr);

tree[k] = tree[2 * k] + tree[2 * k + 1];
}

long long query(int k, int l, int r, int ql, int qr)
{
    if (ql <= l && r <= qr)
        return tree[k];

    push(k, l, r);

    int mid = (l + r) / 2;
    long long ans = 0;

    if (ql <= mid)
        ans += query(2 * k, l, mid, ql, qr);

    if (qr > mid)
        ans += query(2 * k + 1, mid + 1, r, ql, qr);

    return ans;
}

int main()
{
    int n, q;

    scanf("%d %d", &n, &q);

    for (int i = 1; i <= n; i++)
        scanf("%lld", &arr[i]);
    printf("\nOutput:\n");
    build(1, 1, n);
    while (q--)
    {
        int type, a, b;

        scanf("%d %d %d", &type, &a, &b);
        if (type == 1)
        {
            update(1, 1, n, a, b);
        }
    }
}
```

```
else
{
printf("%lld\n", query(1, 1, n, a, b));
}
}

return 0;
}
```

Output:

Test 1:

```
○ aadityajaiswal@Aad
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
Output:
9
Output:
```

Test 2:

```
32
● aadityajaiswal@Aadityas
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
Output:
17

32
○ aadityajaiswal@Aadityas
```